

ASG AIRLINES

END-TO-END DATA ENGINEERING CASE STUDY

Neo Stats-Round 2

Prepared by: Amogh Venkat D
Programme / Course: MCA B
Institution: Christ (Deemed to be University)
Submission Date: 06/09/2026

Documents Attached

Item	Details
Project	ASG Airlines Case Study
Source workbook	UseCase - Airlines.xlsx
Cloud platform	Microsoft Azure
Storage	Azure Data Lake Storage Gen2
Orchestration	Azure Data Factory
Processing	Azure Databricks / PySpark
Reporting	Power BI

Contents

- Executive Summary
- The Business Need Behind the Pipeline
- What the Source Workbook Contains
- What the Initial Profiling Revealed
- The Azure Architecture and Why It Fits the Case
- How the Data Lake Is Organized
- Moving the Workbook into Azure
- Turning Raw Records into Trusted Data
- How Invalid Data Is Handled
- The Overnight Flight Edge Case
- Keeping Passenger Information Out of Analytics
- The Analytical Model Behind the Dashboard
- Measures That Turn Data into Airline KPIs
- Power BI as the Business-Facing Layer
- Findings from the Processed Dataset
- Operational Reliability and Monitoring
- Designing for Larger Airline Workloads
- Assumptions and Practical Limitations
- Where the Platform Can Go Next
- Conclusion
- Appendix: Evidence and Submission Checklist

1. Executive Summary

This project develops an end-to-end cloud data engineering solution for ASG Airlines using the supplied UseCase - Airlines.xlsx workbook. The workbook contains operational data for flights, bookings, payments and passengers, but the source records are not immediately suitable for reliable reporting. The engineering task is therefore not simply to load the spreadsheet into a dashboard; it is to create a controlled path from raw records to trustworthy business information.

The proposed platform uses Azure Data Lake Storage Gen2 as the central storage layer, Azure Data Factory as the orchestration layer, Azure Databricks with PySpark for data validation and transformation, and Power BI for reporting. The architecture separates raw, cleaned, analytical and rejected records through Bronze, Silver, Gold and Quarantine zones. This makes the pipeline easier to audit and gives data-quality problems a visible destination instead of hiding them.

The supplied data contains several real-world inconsistencies. Flights include missing and UNKNOWN airline values, 6F flight prefixes that are treated as corrupted 6E identifiers, exact duplicate records and an unusual timestamp case. Bookings contain missing and INVALID statuses, payments contain missing and INVALID amounts, and the passenger dataset contains duplicate identifiers and sensitive identity and contact information. The implemented rules standardize these issues where a reliable correction is possible and quarantine records that cannot safely be used.

After processing, 1,005 flight records were retained in the analytical flight fact and 15 flight records were quarantined. The processed flight data contains four standardized airlines, 30 unique routes and one overnight flight. The average processed flight duration is approximately 164.62 minutes. The reporting design intentionally excludes names, Aadhaar numbers, passport numbers, phone numbers, email addresses and emergency-contact information from Power BI.

2. The Business Need Behind the Pipeline

Airline data has value only when operational records can be converted into consistent information. A flight identifier that changes format, an airline field containing UNKNOWN, or a negative duration created by a midnight crossing can all produce misleading metrics if they are passed directly into a reporting system. The same principle applies to bookings and payments: invalid statuses and amounts need to be identified before they influence operational reporting.

The purpose of this case study is to demonstrate how a cloud data engineering pipeline can address those problems systematically. The expected outcome is a reusable flow in which the source data is preserved, transformations are reproducible, exceptions remain traceable and the final reporting layer contains only fields that are necessary for decision-making.

2.1 Objectives

The implementation is designed to ingest all four source datasets, preserve the raw data, standardize and validate the records, separate invalid data, create analytical facts and dimensions, protect passenger PII, and expose useful airline KPIs through Power BI. The design also provides a foundation for future incremental ingestion, operational monitoring and predictive analytics.

3. What the Source Workbook Contains

The source workbook is divided into four worksheets. Flights contains schedule and route information. Bookings connects passengers with flights and includes booking status and travel/contact fields. Payments connects financial transactions to bookings. Passengers contains demographic, identity and contact attributes.

Worksheet	Rows	Columns	Primary role
flights	1,020	7	Flight schedules, airline, route, timestamps and source duration
payments	1,000	4	Payment transactions associated with bookings
bookings	1,000	9	Booking, passenger, flight and status information
passengers	1,039	9	Passenger identity, demographic and contact data

3.1 Flight Data as the Core Analytical Grain

The flight dataset forms the core of the main analytical model. Its fields are flight_id, airline, source, destination, departure_time, arrival_time and duration. The pipeline converts these source attributes into standardized identifiers, timestamps, route labels and a recalculated duration. The resulting flight fact is intentionally compact because the dashboard is designed around flight operations rather than passenger-level PII.

3.2 Relationships Across the Operational Data

Bookings reference both passengers and flights, while payments reference bookings. Profiling found no unresolved booking-to-flight, booking-to-passenger or payment-to-booking references. This means the source workbook has a usable relational structure even though individual fields require cleaning before they can be trusted.

4. What the Initial Profiling Revealed

Profiling was performed before transformation so that the cleaning rules were based on observed source behavior rather than assumptions. The results show a mixture of missing values, explicit invalid markers, duplicate records and formatting inconsistencies.

4.1 Flight Records

Observation	Count / result	Interpretation
Missing airline	41	Airline information is incomplete in the source.
UNKNOWN airline	31	Explicit unknown values require standardization.
Exact duplicate rows	15	Duplicate source records should not inflate flight KPIs.
6F prefix	273	The prefix consistently maps to IndiGo and is standardized to 6E.
Overnight flight	1	A cross-midnight case requires timestamp normalization.
Unique routes after processing	30	The dataset covers 30 distinct source-destination combinations.

The 6F prefix is particularly useful as a data-quality example. Cross-checking the source records showed that 6F records map to IndiGo: 249 already carry the IndiGo airline value, while the remaining records are UNKNOWN or missing. The pipeline therefore treats 6F as a corrupted representation of 6E rather than creating a new airline category.

4.2 Booking and Payment Records

Dataset	Issue	Count
Bookings	Missing status	45
Bookings	INVALID status	30
Payments	Missing amount	48
Payments	INVALID amount	30

These values are not simply removed from the source. They are recorded as quality exceptions so the organization can measure how much of the source data is affected.

4.3 Passenger Records

The passenger worksheet contains 1,039 rows but only 1,000 unique passenger IDs, resulting in 39 duplicate identifiers. Ten records have missing last names. The data also contains Aadhaar, email, phone and other identity fields that are not required for the operational dashboard. The pipeline therefore resolves duplicate candidates deterministically for controlled processing and keeps sensitive attributes out of the Gold reporting layer.

5. The Azure Architecture and Why It Fits the Case

The architecture separates storage, orchestration, processing and visualization so that each service has a clear responsibility. ADLS Gen2 acts as the durable data lake, Data Factory coordinates movement and execution, Databricks performs data engineering logic, and Power BI presents the curated results.

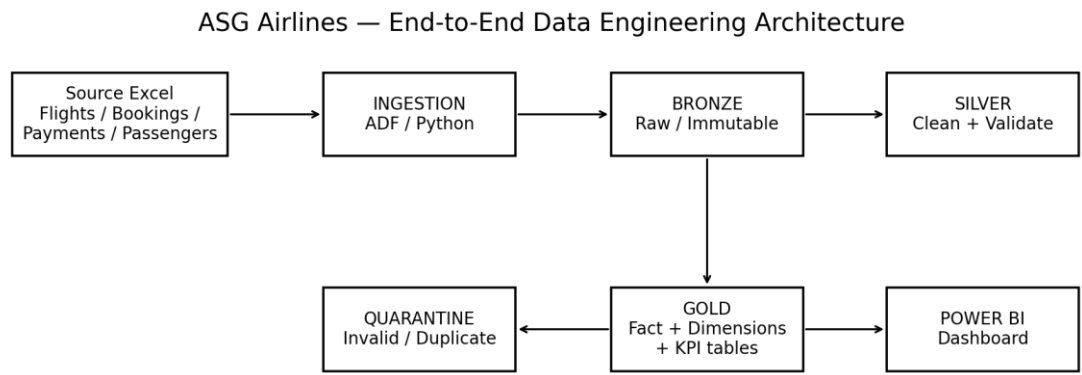
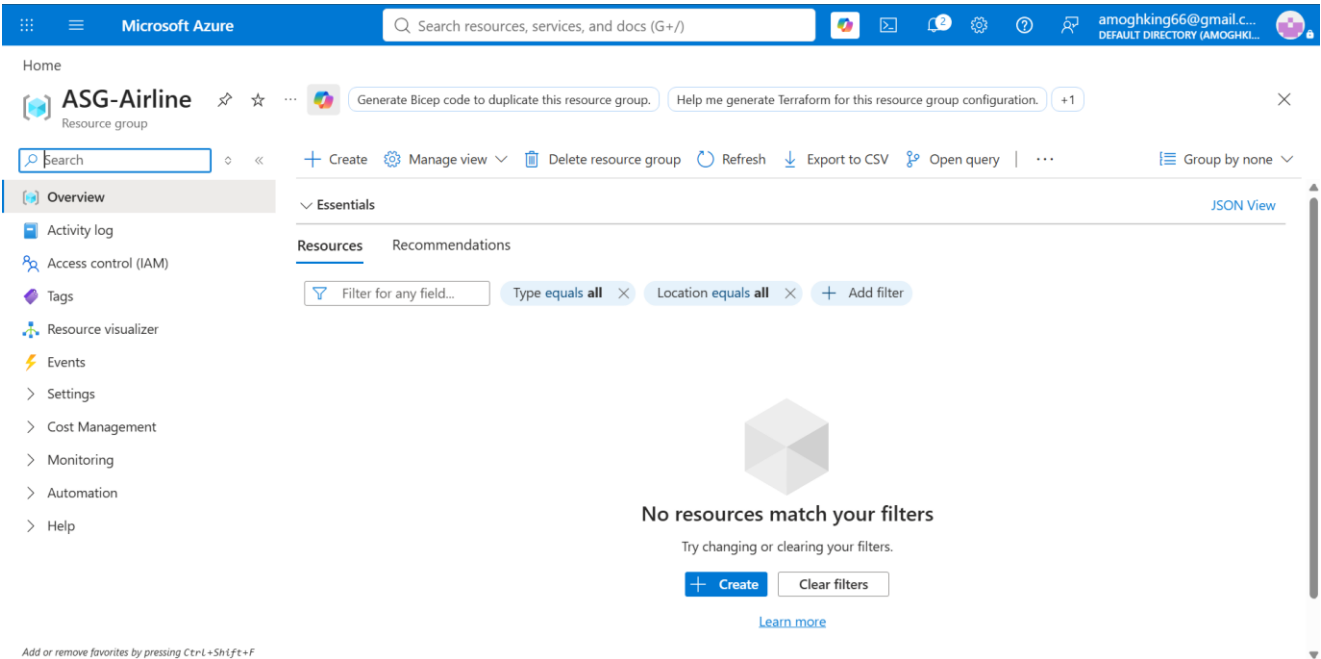


Figure 5.1 — ASG Airlines end-to-end Azure architecture

5.1 Azure Services in the Implementation

Azure component	Responsibility in this project
ASG-Airlines-RG	Keeps the project resources under one logical management boundary.
ADLS Gen2	Stores Bronze, Silver, Gold and Quarantine data.
ASG-Airlines-ADF	Orchestrates workbook ingestion and downstream processing.
ASG-Airlines-DB	Runs PySpark validation and transformation workloads.
Power BI	Provides the business-facing analytical dashboard.



6. How the Data Lake Is Organized

The data lake uses four zones with different purposes. Bronze is the landing area and should preserve what arrived from the source. Silver contains standardized and validated records. Gold contains business-ready tables designed for Power BI. Quarantine contains records that failed defined validation rules. Keeping these zones separate means that cleaning does not destroy the original evidence.

Zone	Purpose	ASG Airlines content
Bronze	Raw landing layer	Flights, bookings, payments and passengers as ingested
Silver	Trusted working data	Standardized and validated datasets
Gold	Reporting layer	Fact, dimensions and KPI aggregates
Quarantine	Rejected records	Invalid or anomalous records with traceability

6.1 Storage Paths

Within the four zones, the datasets are separated into flights, bookings, payments and passengers. Gold uses a more analytical structure with fact_flight, dim_airline, dim_route, dim_date and KPI outputs. This layout keeps operational ingestion paths distinct from reporting structures.

The screenshot displays the Microsoft Azure portal interface. The top navigation bar shows the 'Microsoft Azure' logo and a search bar. The main content area is divided into two sections. The top section, titled 'asgairlines_1788682189865 | Overview', shows a deployment status of 'Your deployment is complete'. It includes details such as the deployment name, subscription (Azure for Students), resource group (ASG-Airline), start time (9/6/2026, 1:40:27 PM), and correlation ID. A 'Go to resource' button is visible. The bottom section, titled 'asgairlines | Containers', shows a list of containers. The left sidebar contains navigation links for Overview, Inputs, Outputs, Template, Access Control (IAM), Data storage, Containers, Settings, Configuration, Endpoints, and Help. The container list table has columns for Name, Last modified, Anonymous access level, Lease state, and Storage connector. The containers listed are \$logs, bronze, gold, quarantine, and silver.

Name	Last modified	Anonymous access level	Lease state	Storage connector
\$logs	9/6/2026, 1:40:54 PM	Private	Available	-
bronze	9/6/2026, 2:22:22 PM	Private	Available	-
gold	9/6/2026, 2:22:38 PM	Private	Available	-
quarantine	9/6/2026, 2:23:01 PM	Private	Available	-
silver	9/6/2026, 2:22:32 PM	Private	Available	-

7. Moving the Workbook into Azure

Azure Data Factory is used as the orchestration layer. Because the original input is an Excel workbook, a practical ingestion pattern is to extract each worksheet into a simple cloud-friendly format such as CSV or Parquet before Databricks performs the heavier transformations. This keeps the orchestration layer focused on movement and the processing layer focused on data engineering.

7.1 Pipeline Execution

The pipeline begins with the Excel source and lands four datasets in Bronze. Once ingestion completes, the Databricks transformation job is triggered. Databricks reads Bronze data, applies validation and standardization rules, writes accepted records to Silver, writes rejected records to Quarantine and publishes Gold outputs. Power BI then consumes the Gold layer.

1. Create the ADLS Gen2 linked service in Azure Data Factory.
2. Configure the Excel source and the four worksheet extracts.
3. Create Bronze sinks for flights, bookings, payments and passengers.
4. Add the Databricks activity after the ingestion activities.
5. Run the pipeline and verify activity status and row counts.
6. Publish the validated Silver, Gold and Quarantine outputs.

In a production implementation, the pipeline would also carry parameters such as source date, file name and processing run ID. That would make the same pipeline reusable for daily or incremental airline data.

The screenshot displays the Azure Data Factory (ADF) console. On the left, the 'Factory Resources' pane shows a list of pipelines and datasets. The 'pipelines2' pipeline is selected. The main area shows the 'Activities' tab for 'pipeline2', which includes a 'Copy data' activity. The 'Properties' pane on the right shows the 'General' tab for the 'Copy data' activity, with the name 'PL_ASG_Passengers_Bronze'. The 'Output' tab at the bottom shows the pipeline status as 'Succeeded' with a green checkmark. The pipeline ID is 'da83c084-8f44-4e34-b50c-3471804a7a98'. The 'run ID' is '50c-3471804a7a98'. The 'Pipeline status' is 'Succeeded'. The 'All status' dropdown is set to 'Monitor in Azure Metrics'. The 'Showing 1 - 1 of 1 items' is displayed. The 'Activity name' is 'Copy data1'. The 'Activity status' is 'Succeeded'. The 'Activity output' is 'Copy data1'.

8. Turning Raw Records into Trusted Data

Databricks is where the business rules become executable transformations. The flight pipeline starts by trimming and uppercasing identifiers, standardizing the 6F prefix to 6E and replacing UNKNOWN airline values with nulls so that a controlled inference rule can be applied. When the flight prefix identifies an airline unambiguously, the airline name is filled from the mapping.

The timestamp transformation is equally important. Departure and arrival values are parsed into timestamp types rather than treated as strings. If an arrival timestamp is earlier than departure, the record is interpreted as a cross-midnight flight and the arrival is advanced by one day. Duration is then recalculated from the normalized timestamps. This is safer than trusting the source duration field because it creates one consistent definition of flight duration.

8.1 Standardization Rules

Source condition	Transformation
6F flight prefix	Standardize to 6E
UNKNOWN or blank airline	Treat as missing, then infer from prefix
Lowercase or padded identifiers	Trim and uppercase
Source / destination formatting	Standardize to uppercase
Arrival before departure	Add one day and mark overnight
Source duration	Recalculate from corrected timestamps
Duration <= 0 or > 480 minutes	Flag as an anomaly

The resulting fact_flight table contains flight_id, airline, source, destination, route, departure_datetime, arrival_datetime, flight_duration_minutes, flight_duration_hours, is_overnight and duration_anomaly. This is the dataset that ultimately drives the main flight KPIs.

The screenshot displays the Databricks workspace interface. On the left, a sidebar contains navigation options like 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', 'Marketplace', 'SQL', 'SQL Editor', 'Queries', 'Dashboards', 'Genie Agents', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', and 'Runs'. The main area shows a notebook titled 'NB_ASG_Airlines_Silver' with a code cell that has been executed 4 minutes ago. The code uses `display(dbutils.fs.ls("abfss://bronze@asgairlines.dfs.core.windows.net/flights/"))` to list files. Below the code, a table view shows the results of the file listing operation. The table has four columns: 'path', 'name', 'size', and 'modificationTime'. It contains two rows of data, both representing 'UseCase - Airlines.xlsx' files in different formats (csv and txt). The bottom of the interface shows a status bar indicating '2 rows' and an '8.849s runtime'.

path	name	size	modificationTime
abfss://bronze@asgairlines.dfs.core.windows.net/flights/UseCase - Airlines.xlsx.csv	UseCase - Airlines.xlsx.csv	123580	178868696000
abfss://bronze@asgairlines.dfs.core.windows.net/flights/UseCase - Airlines.xlsx.txt	UseCase - Airlines.xlsx.txt	123580	178868600700

9. How Invalid Data Is Handled

The pipeline follows a quarantine-first approach for records that cannot safely participate in analytics. A flight identifier must match the expected airline prefix and three-digit number pattern. Departure and arrival timestamps must parse correctly, and the recalculated duration must be greater than zero and no more than eight hours under the project rule. Exact duplicate flight records are not allowed to inflate the analytical fact.

Booking status is accepted only when it is CONFIRMED, CANCELLED or PENDING. Missing and INVALID values are treated as quality failures. Payment amounts must be numeric and non-negative. Passenger duplicates are not treated as ordinary row-level duplicates because the same passenger ID can occur with different levels of completeness; instead, a deterministic quality score is used to choose the stronger record.

9.1 Validation Outcomes

Dataset	Validation approach	Destination for failed records
Flights	Identifier, timestamp, duration and duplicate checks	quarantine/flights
Bookings	Status validation	quarantine/bookings
Payments	Numeric and non-negative amount validation	quarantine/payments
Passengers	Duplicate-ID candidate review and safe projection	quarantine/passengers / restricted processing

The purpose of quarantine is not merely technical. It gives the project an audit trail: an invalid record remains available for inspection, while the Gold layer remains protected from known quality failures.

10. The Overnight Flight Edge Case

One of the most useful data-engineering decisions in this project concerns a flight whose arrival timestamp occurs before its departure timestamp. For an airline schedule, that can be a legitimate cross-midnight flight rather than a negative-duration flight. Treating every earlier arrival as invalid would therefore remove potentially valid operational data.

The pipeline checks the two timestamps after parsing. When arrival is earlier than departure, it sets `is_overnight` to 1 and adds one calendar day to the arrival timestamp. Duration is then recalculated from the corrected timestamps. This produces a transparent rule that can be tested and explained to a reviewer.

Field	Rule applied
<code>is_overnight</code>	1 when arrival initially occurs before departure
<code>arrival_datetime</code>	Advance by one day for an overnight record
<code>flight_duration_minutes</code>	Corrected arrival minus departure
<code>duration_anomaly</code>	1 when corrected duration is ≤ 0 or > 480 minutes

The source workbook also contains an unusual SJ192 timestamp/duration case in which the arrival appears on the previous calendar date and the supplied duration is inconsistent. Because this case does not behave like a normal cross-midnight schedule, it is treated as an anomaly rather than being silently corrected into a plausible flight.

11. Keeping Passenger Information Out of Analytics

The passenger and booking sheets contain personal information that is not needed to answer the primary operational questions in this case study. A responsible analytics design therefore minimizes the data exposed to the reporting layer. Names, Aadhaar IDs, passport numbers, phone numbers, email addresses and emergency-contact details remain outside the Gold dashboard model.

For controlled processing where a stable identifier is required, the project creates a SHA-256 hash of the passenger identifier. The safe analytical projection can retain attributes such as age and gender without exposing direct identity fields. This provides a practical balance between analytical usefulness and data minimization.

11.1 Security in the Azure Environment

The Azure implementation should use managed identity or an Azure Access Connector for storage access rather than embedding storage keys in notebooks. Role-based access control should follow least privilege, with Storage Blob Data Contributor assigned only to identities that genuinely need write access. Where secrets are unavoidable, Azure Key Vault should be used. Screenshots included in the final report must not expose keys, tokens, connection strings or personal data.

12. The Analytical Model Behind the Dashboard

The Gold layer uses a compact analytical model rather than reproducing every operational source field. Fact_Flight is the central table because the main business questions concern flight volume, duration, routes and overnight operations. Dim_Airline, Dim_Route and Dim_Date provide descriptive context and make filtering in Power BI straightforward.

Gold table	Purpose	Representative fields
Fact_Flight	Flight-level analytical facts	flight_id, airline, route, departure_datetime, arrival_datetime, duration
Dim_Airline	Standardized airline reference	airline
Dim_Route	Source-destination reference	source, destination, route
Dim_Date	Calendar analysis	flight_date, day, month, year
Airline_KPI	Airline-level aggregates	total_flights, avg_duration, overnight_flights, anomalies
Route_KPI	Route-level aggregates	total_flights, avg_duration, overnight_flights, anomalies
Daily_KPI	Daily operational trend	flight_date, total_flights, avg_duration, overnight_flights

The resulting model is small enough to be easy to understand and large enough to support the required dashboard. In a production implementation, the same structure could be extended with actual flight movements, delays, cancellations and airport dimensions.

13. Measures That Turn Data into Airline KPIs

The dashboard is centered on a small set of measures that directly describe the processed flight population. Total Flights counts validated analytical records. Average Duration summarizes the normalized duration in minutes. Overnight Flights counts cross-midnight operations, while Overnight % expresses them relative to total flight volume. Anomalies and Anomaly % expose data-quality exceptions that remain in the analytical model when a flagged record is intentionally retained.

Measure	Definition
Total Flights	Number of rows in Fact_Flight
Average Duration	Average flight_duration_minutes
Overnight Flights	Sum of is_overnight
Overnight %	Overnight Flights / Total Flights
Anomalies	Sum of duration_anomaly
Anomaly %	Anomalies / Total Flights

13.1 Power BI DAX

Total Flights = COUNTROWS(Fact_Flight)

Average Duration = AVERAGE(Fact_Flight[flight_duration_minutes])

Overnight Flights = SUM(Fact_Flight[is_overnight])

Overnight % = DIVIDE([Overnight Flights], [Total Flights])

Anomalies = SUM(Fact_Flight[duration_anomaly])

Anomaly % = DIVIDE([Anomalies], [Total Flights])

14. Power BI as the Business-Facing Layer

Power BI is the final presentation layer and should connect to Gold rather than directly to the raw workbook. The dashboard can therefore be refreshed without changing the underlying data-quality rules, and the user sees a consistent definition of each KPI.

14.1 Executive View

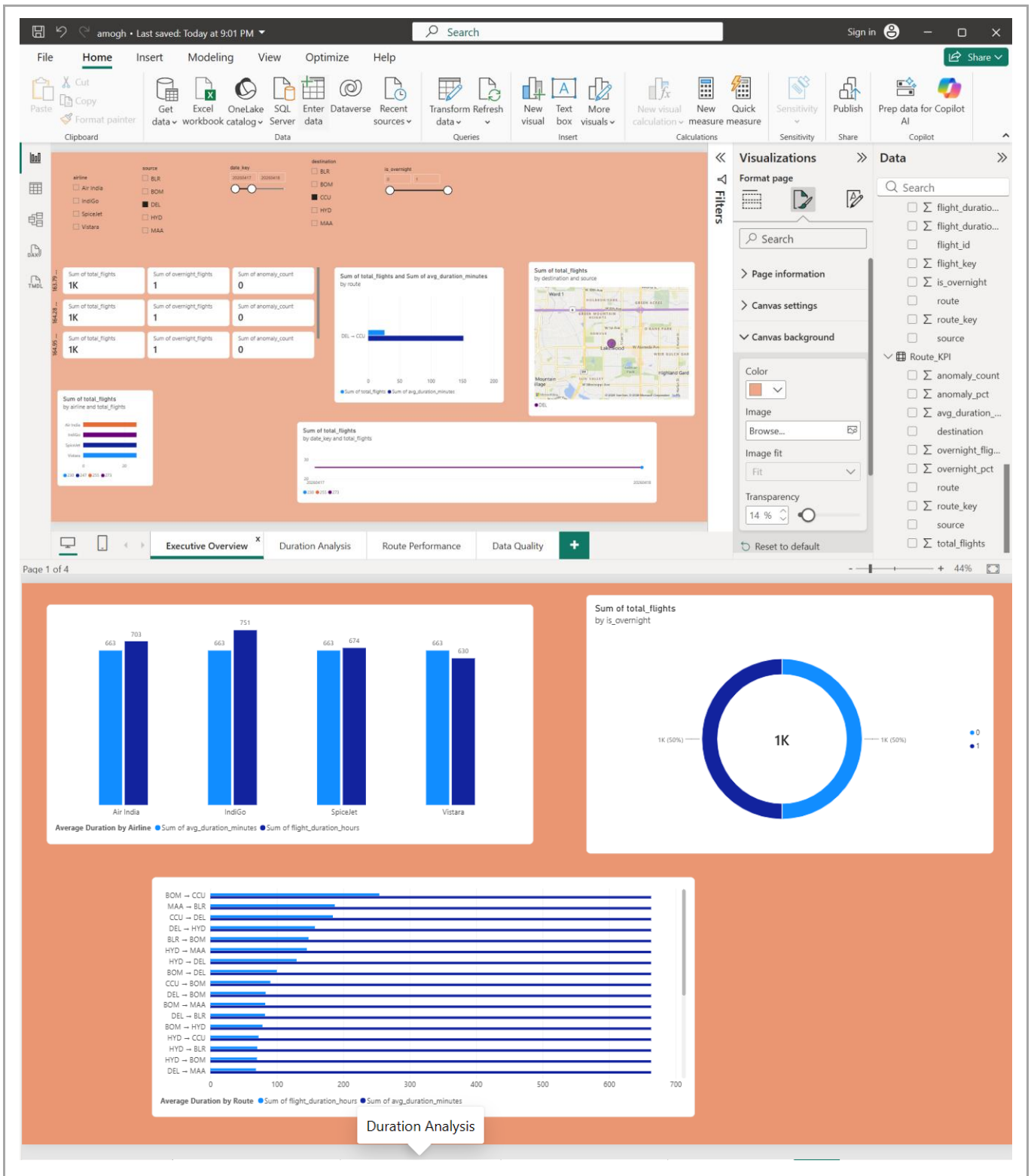
The first page should answer the basic operational questions immediately: how many flights are in the validated population, what is the average duration, how frequently do flights cross midnight, and how many anomalies are present. Airline and route visuals provide context, while a daily trend shows how flight activity changes across the available dates.

14.2 Duration and Route Analysis

A second page can compare average duration across airlines and routes and distinguish normal from overnight operations. A route-focused page can then show which source-destination pairs carry the most flight volume and how their average durations differ.

14.3 Data Quality View

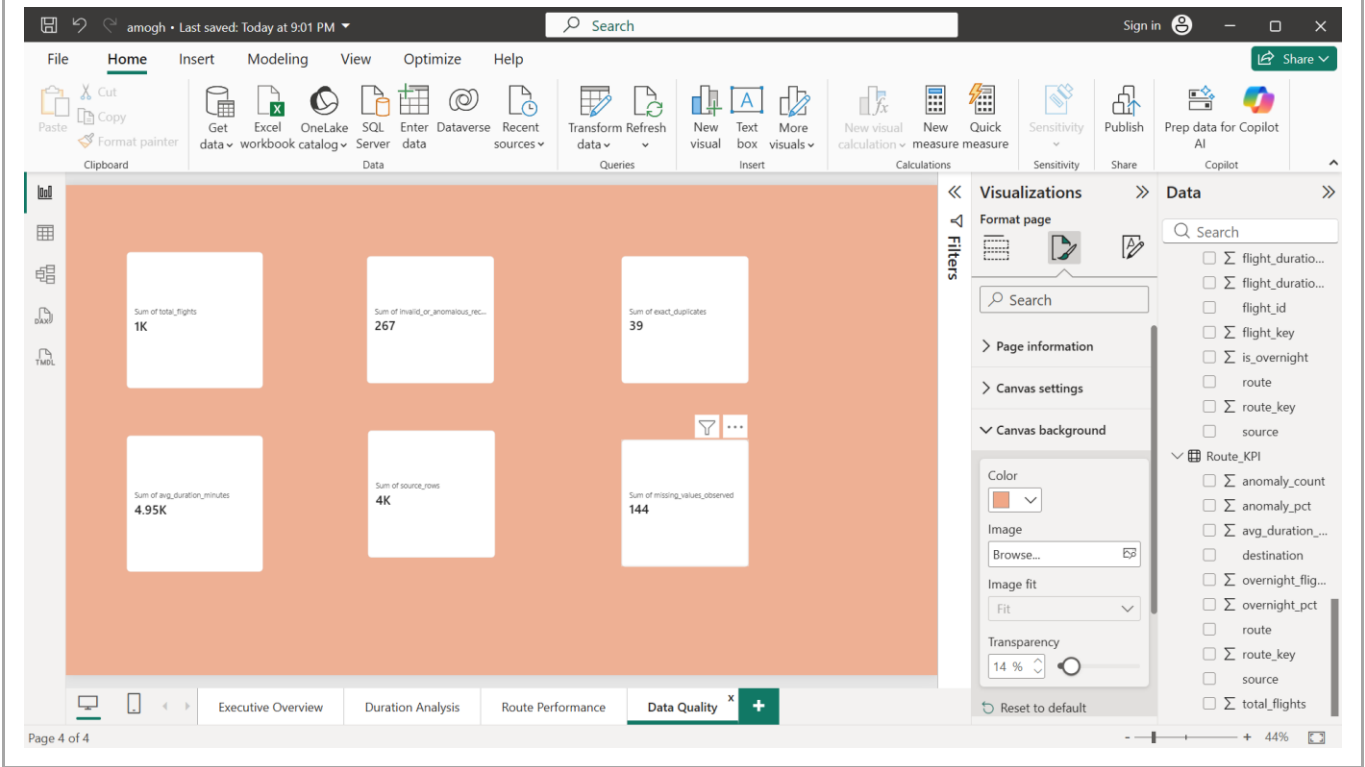
The final page should make data quality visible. It can show valid versus quarantined records, invalid booking statuses, invalid payment amounts, duplicate passenger IDs and flight anomalies. This is important because the data engineering pipeline itself is one of the main deliverables of the case study.



Route Traffic Matrix								
source	BLR	BOM	CCU	DEL	HYD	MAA	Total	
BLR			60	21	19	19	16	135
BOM		23		90	39	26	27	205
CCU		20	33		72	21	24	170
DEL		29	28	26		54	23	160
HYD		26	27	26	42		57	178
MAA		65	21	24	26	21		157
Total		163	169	187	198	141	147	1005

Route Metrics Table				
route	avg_duration_minutes	anomaly_count	overnight_flights	
BLR → BOM	163.79	0	1	
BLR → CCU	163.79	0	1	
BLR → DEL	163.79	0	1	
BLR → HYD	163.79	0	1	
BLR → MAA	163.79	0	1	
BOM → BLR	163.79	0	1	
BOM → CCU	163.79		1	

Route Performance



15. Findings from the Processed Dataset

The processed results provide a concrete measure of what the engineering work changed. The analytical flight fact contains 1,005 records, while 15 source flight rows were quarantined. The average processed flight duration is 164.62 minutes, the data contains 30 unique routes and one overnight flight. The standardized airline population consists of Air India, IndiGo, SpiceJet and Vistara.

Metric	Processed result
Analytical flight rows	1,005
Flight quarantine rows	15
Average flight duration	164.62 minutes
Unique routes	30
Overnight flights	1
Standardized airlines	Air India, IndiGo, SpiceJet, Vistara
Invalid payment amounts	30
Missing or INVALID booking statuses	75
Duplicate passenger IDs	39
Missing passenger last names	10

These figures also show why profiling should happen before dashboard development. If duplicate flights, inconsistent identifiers or malformed durations were passed directly into Power BI, the resulting KPIs would represent source-system defects as though they were business activity.

16. Operational Reliability and Monitoring

A production version of this pipeline should make each execution observable. The ingestion stage should record the source file, worksheet, ingestion time and row count. The transformation stage should record accepted and rejected rows and the rules that caused rejection. Gold publication should record output paths and row counts, while Power BI refreshes should be monitored separately.

A useful operational safeguard is to compare current row counts with historical expectations. A sudden drop in flight records, for example, could indicate an upstream source problem rather than a legitimate business change. Likewise, a sudden increase in quarantined payments should be visible to the team responsible for the source system.

Pipeline stage	Information worth recording
Ingestion	File, worksheet, timestamp and source row count
Validation	Valid count, invalid count and rejection reasons
Transformation	Processing duration and transformed row count
Gold publishing	Output location, row count and completion status
Reporting	Power BI refresh status and timestamp

Critical failures such as a missing source file should stop the run. Individual record failures can normally be isolated into quarantine so that one malformed row does not prevent valid data from being processed.

17. Designing for Larger Airline Workloads

The supplied workbook is small, but the architecture is designed to scale beyond a classroom-sized dataset. At larger volumes, Silver and Gold should use Parquet or Delta Lake rather than CSV because columnar storage and transactional capabilities are better suited to repeated analytical workloads.

Incremental processing would become important once daily or hourly airline data is introduced. ADF could pass ingestion dates or file identifiers to Databricks, allowing only new records to be processed. Fact data could be partitioned by flight date, while Power BI could use incremental refresh for large historical models.

The hardcoded airline-prefix mapping is acceptable for this case study because the mapping was established from the supplied data. A production platform should move this mapping into a governed reference table. The same approach could be applied to airports, route classifications, airline codes and other business reference data.

18. Assumptions and Practical Limitations

Several assumptions are necessary because the source workbook does not include a separate master-data system. The 6F prefix is treated as a corrupted 6E identifier because its records consistently correspond to IndiGo. AI, 6E/6F, SJ and UK are mapped to Air India, IndiGo, SpiceJet and Vistara respectively. These mappings should be replaced with an authoritative reference table if the solution is moved into production.

An arrival timestamp earlier than departure is interpreted as a cross-midnight flight unless the surrounding data indicates that the record is malformed. Duration is recalculated from normalized timestamps rather than trusted from the source duration field. Durations above eight hours are treated as anomalies for this project; this threshold is a practical case-study rule, not a universal airline standard.

The main limitation is that the workbook is static. The case study demonstrates the architecture and transformation logic, but it does not represent a live airline event stream. It also does not contain actual departure delays, arrival delays or aircraft operational events, so predictive delay analysis is outside the current scope.

20. Conclusion

The ASG Airlines case study demonstrates the complete journey from an inconsistent operational workbook to a structured cloud analytics platform. The central design decision is to separate raw data from validated and business-ready data so that transformation does not destroy traceability.

Azure Data Lake Storage Gen2 provides the storage foundation, Azure Data Factory manages orchestration, Azure Databricks applies the data-quality and transformation rules, and Power BI exposes the final metrics. The solution also treats passenger privacy as part of the data-engineering design rather than as an afterthought.

The resulting platform is small enough to understand clearly and structured enough to extend. Most importantly, it makes data-quality decisions explicit: malformed records are not hidden, corrections such as the 6F-to-6E standardization are explainable, overnight flights are handled deliberately, and sensitive passenger attributes are kept outside the reporting layer.

